

Project Topic

Your Name

March 4, 2026

1 Introduction

Traffic management remains a crucial issue in large cities, especially rapidly developing cities without extensive urban planning. Poor traffic management results in congestion, unnecessary emissions, and accidents that impact the safety of the city's residents. In order to manage the flow of traffic within a city, many cities turn to data-driven approaches that inform urban planning.

"Smart cities" refer to cities that utilize data and computational techniques to improve the functioning of a city. For example, a smart city may use location services on cell phones to determine the best times and places to collect waste within a city. Intelligent traffic systems, a crucial component of smart cities, aggregate and analyze data to manage traffic within a city. Intelligent traffic systems may make use of traffic data, such as data gathered from Uber or from sensors placed around the city, to model traffic in real time. These traffic models can then be used to create software that responds to current traffic flows and decreases congestion, such as software like Google Maps that re-routes traffic to less congested areas. Urban planners, public sector transportation employees, politicians, and everyday people rely on accurate traffic models to make informed decisions about new transit routes or traffic policy.

Traffic accident analysis is an important subsection of traffic management. Motor vehicle crashes are the top cause of death in the United States among people ages 1-44, with traffic fatalities causing about 44,000 deaths in 2022. [?] Traffic accident modeling utilizes traffic accident data to determine when and where traffic accidents will occur. This data can be used by city planners to create policy geared towards decreasing traffic accidents. For example, [throw in an example of how policymakers use traffic data to make decisions about traffic, like using smartphone or something.] Intelligent traffic systems additionally use traffic models to reroute traffic away from accident-prone areas.

The majority of traffic accident prediction studies focus on determining merely whether or not an accident will happen, but leave out additional analysis crucial to urban planners or policymakers that help these people to make informed traffic planning decisions. Traffic accident models which solely predict accidents do not explain how or what factors led into the algorithm deciding the result, which is both uninformative and seen as unreliable by policymakers who don't understand how the result came to be. Thus, many traffic studies rely on explanatory models to analyze the underlying factors that contribute to an accident or the severity of the accident. Additionally, the majority of traffic accident models focus

on large cities where data is more readily available, rather than small cities where different factors may contribute to accident occurrence and severity.

This study aims to provide better insight into traffic accidents in small American towns, including factors that can influence the likelihood and severity of a traffic accident. Using XGBoost analysis, this study will analyze a dataset of roughly 10,000 traffic accidents in Wooster, Ohio from 2021-2026 to model the likelihood of accidents, the severity of the accident, and determine which factors most contributed to causing a traffic accident. The results from this study will provide insight into what factors cause traffic accidents, where these accidents, providing information to traffic officials on strategic decisions that can be made to enhance public safety.

2 Background

The decision tree machine learning models of XGBoost and Random Forest are frequently used to model traffic in the literature. Decision trees are simple machine learning algorithms that split into smaller, more specific branches, and come to a decision by tracing a path down the tree. In order to figure out which path to go down, the decision tree can be thought of as answering a series of questions. As an example, a decision tree may have been created to decide whether or not a student should study. The first branch may ask, "What time is it?" The tree may branch into two separate paths: one for if the time is between 11 pm and 6 am, and one for if it is 6am-11pm. Then, it may ask another question, "Do you have an exam tomorrow?" Here, the tree will split off into four paths: One for if it is nighttime and there is an exam, one for if it is nighttime and there is not an exam, one for if it is daytime and there is an exam, and one for if it is daytime and there is not an exam tomorrow. The decision tree may say that you should study if it is daytime and you have an exam, but not under any other condition.

In order for the decision tree to decide where to branch (say, maybe the first question should split at 5 am and 12 pm rather than 6 am and 11 pm), the trees are trained on a large body of data. The difference between XGBoost and Random Forest lies in the way that each new tree decision tree is created. Random Forest follows the "bagging" technique, which means that each tree is trained individually on a different subset of the data. XGBoost, in contrast, follows the "boosting" technique, wherein trees are trained sequentially, with the newest tree correcting the errors of the previous tree. In both training techniques, each tree will come to a slightly different conclusion. The model produces a single, final result by summing the result that each tree in the model comes to.

In order to optimize decision tree models, the researcher must determine specific parameters, such as how many times the decision tree should be allowed to branch or how many decision trees the model should have. Too many trees results in the risk of overfitting the model, whereas too few trees will not fit the model closely enough to the data and leave a wide margin of error. Bayesian optimization techniques are used to determine the parameters that the model should use for optimal accuracy.

3 Related Works

A long history of data-driven traffic models that have been used to make planning decisions. Data visualization methods plot a simple database of accident data onto a map to give the data an easily visualized, geospatial component. Other visualization models utilize BI graphics and tables to display accident causes or traffic metrics. These approaches cleanly visual historical accident data but do not model or predict future accidents.

The traffic management field has relied on statistical regression analysis as a predictive model in the past, such as historical averages or ARIMA. Historical averages take the average of all data over a certain time period and use that value as a prediction of what will happen in the future. ARIMA, or Autoregressive Integrated Moving Average, provides a more complex analysis by integrating three components: "Autoregressive," which computes the current volume of traffic based on previous traffic counts (ex. if traffic is high at 10 am, it will likely still be high at 10:10 am), "Integrated," which handles broad trends over long periods of time, and "Moving Average," which allows the model to correct itself based on its past errors. These methods are computationally simple, but their simplicity limits their ability to handle non-linear relationships or the incorporation of additional variables, such as weather. ARIMA, specifically, handles short-term time-based predictions well, but struggles with real-time traffic modeling, managing the impact of traffic flows from multiple locations, or handling data with a large variety of variables.

Recent approaches using machine learning and neural networks produce models of traffic flows, jams, and accidents have shown a marked increase in accuracy of traffic modeling compared to traditional statistical methods. A large body of literature showcase the various ways machine learning has been used to model traffic flows. One study in Thailand models traffic flows and car sales to track emissions across the city, while another study models traffic to create a signal traffic control system in Ali Mendjeli, Algeria. Others model traffic jams, accident prediction, and travel time, but most traffic studies share similar methods, machine learning techniques, and limitations in the creation of their traffic models.

A plurality of studies have observed the superior performance of machine learning models, especially decision tree algorithms, over neural networks such as LSTM or CNNs for modeling tabular traffic data. Random Forest and XGBoost have similar accuracy, with some comparison studies finding an edge when using XGBoost and some finding Random Forest to be the most efficient. Decision tree models often do not create real-time traffic maps, but perform especially well on tabular data that is often available for modeling problems. K Nearest Neighbors (KNN) is another frequently used machine learning model used to analyze traffic.

Machine learning models suffer from a lack of interpretability—the model simply reads out a number without speaking on the factors that went into that decision. An XGBoost model may say that there will not be an accident on Beall Avenue tomorrow, but it will not be able to say that it made that decision because the weather will be clear, or because accidents occur less often on Wednesdays, or because it is springtime. In order to create interpretable results, studies use local interpretable methods like SHapley’s Additive exPlanation (SHAP), as well as global feature importance methods like Permutation Feature Importance (PFI) and Feature Importance Ranking Measures (FIRM). Local feature importance conveys information about why the model made each specific decision, whereas global feature importance

explains which features are most important across the entire dataset. However, the effectiveness of each interpretability method may vary based on the machine learning method used, as Abdurashid [1] noted that each feature importance model gave different results when used for each different machine learning method.

Neural networks perform well when performing real-time traffic, such as real time traffic signal decision making. Long Short-Term Memory (LSTM) and Convolutional Neural Network (CNN) are commonly used and handle spatiotemporal data better than machine learning models.

Traffic modeling studies share limitations and concerns. In any smart city project, privacy becomes an issue as personal travel data is being used to create traffic flows. Data privacy concerns contribute to the already herculean difficulty in acquiring the necessary data to create traffic models. Even when data is available, the data may not accurately represent the actual traffic patterns in the city. For instance, many studies use data acquired from Uber or Waze to create neural network-based models, but acknowledge that training a model based on only one transportation platform may be biased since it would not represent the traffic of all individuals within the city. Finally, data availability varies between cities and regions, which results in more data-rich areas receiving more researching resources. Urbanized cities in North America have a larger abundance of data available to train machine learning models or neural networks, such as Uber data, sensors, cameras, and satellite data. Developing cities face greater challenges in acquiring the data required to model traffic flows, and research projects often require collaboration with local governments in order to place sensors or otherwise acquire traffic data that can be used for analysis.

4 Methodology

Dijkstra’s algorithm was chosen for single-source shortest path computation on graphs with non-negative edge weights, following the implementation presented in CLRS [?].

5 Results

An exemplary figure is shown in Figure 1. Any time a figure is used, it must have a caption and be directly referred to and discussed in the text.

6 Conclusion

References

- [1] ABDULRASHID, I., CHIANG, W.-C., SHEU, J.-B., AND MAMMADOV, S. An interpretable machine learning framework for enhancing road transportation safety. *Transportation Research Part E: Logistics and Transportation Review* 195, 0 (2025).

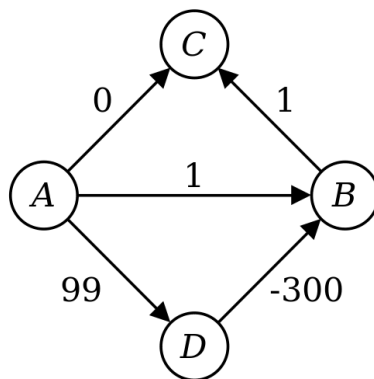


Figure 1: Dijkstra's algorithm does not work on graphs with negative edge weights, as evidenced here.